



END-TO-END REAL-TIME INGESTION AND PROCESSING PIPELINE: BANKING TRANSACTIONS

Assignment



MAY 17, 2026

Fadi Al-Ariqi

Abstract

This document provides a comprehensive technical overview of an end-to-end real-time data ingestion and processing pipeline designed for banking transactions. The pipeline leverages Apache NiFi for data flow orchestration, Apache Kafka for high-throughput message queuing, and Hadoop HDFS for distributed storage. It addresses challenges such as data anomalies, diverse data formats, and real-time processing requirements, ensuring data integrity and scalability for critical financial operations.

1. Executive Summary

The project focuses on establishing a robust and scalable data pipeline for **real-time banking transactions**. This scenario involves tracking live account movements, transfers, and ATM logs, which are critical for fraud detection, real-time analytics, and operational monitoring. The pipeline is engineered to handle a continuous stream of transaction data, including deliberate anomalies to simulate real-world data quality challenges, ensuring the system's resilience and data validation capabilities.

2. Python Streaming Script: Data Generation with Anomalies

A Python streaming script operates in the background, simulating real-time transaction data by writing CSV files into the `./my_data/input` directory every 2-4 seconds. This script is designed to introduce various data quality issues to rigorously test the pipeline's robustness and error-handling mechanisms. The injected anomalies include:

- **Missing customer_id values:** Simulating incomplete transaction records.
- **30% duplicated data:** Introducing redundant entries to test deduplication or idempotent processing.
- **Invalid numeric values:** Including negative amounts and string words like 'invalid' in numeric fields.
- **Four distinct timestamp formats:** Challenging the pipeline's ability to parse and standardize diverse date/time representations.
- **Completely corrupted rows:** Such as '###' or 'NaN', to test the system's ability to gracefully handle severely malformed data.

Python Streaming Script:

```

import csv
import time
import random
import os
from datetime import datetime

OUTPUT_DIR = r"./my_data/input"

os.makedirs(OUTPUT_DIR, exist_ok=True)

customers = ["CUST001", "CUST002", "CUST003", "CUST004", None]
statuses = ["SUCCESS", "FAILED", "PENDING", "UNKNOWN"]

def random_timestamp():
    formats = [
        "%Y-%m-%d %H:%M:%S",
        "%d/%m/%Y %H:%M",
        "%Y/%m/%d",
        "%m-%d-%Y %H:%M:%S"
    ]
    fmt = random.choice(formats)
    return datetime.now().strftime(fmt)

def generate_record():
    return [
        random.randint(1000, 9999),           # transaction_id
        random.choice(customers),             # customer_id
        random.choice([100, 200, 500, -50, "invalid"]), # amount
        random_timestamp(),                   # timestamp
        random.choice(statuses)               # status
    ]

def generate_corrupted_row():
    corrupted_examples = [
        ["###", "???", "NaN"],
        ["", "", ""],
        ["BROKEN_DATA"],
        ["1234", None, "oops", "bad_timestamp"]
    ]
    return random.choice(corrupted_examples)

def create_csv_file():
    timestamp_str = datetime.now().strftime("%Y%m%d_%H%M%S")
    file_name = f"transaction_{timestamp_str}.csv"
    filepath = os.path.join(OUTPUT_DIR, file_name)

    records = []

    for _ in range(random.randint(5, 15)):
        records.append(generate_record())

    if random.random() < 0.3:
        records.append(records[0])

    if random.random() < 0.3:
        records.append(generate_corrupted_row())

    with open(filepath, "w", newline="") as f:
        writer = csv.writer(f)

        writer.writerow(["transaction_id", "customer_id", "amount", "timestamp", "status"])

        for r in records:
            writer.writerow(r)

    print(f"- Generated file: {filepath}")

while True:
    create_csv_file()
    time.sleep(random.randint(2, 4))

```

3. Process Group 1: Ingestion Pipeline

The first stage of the NiFi workflow, named "Assignment 2 (Nifi, Kafka, Hadoop)", is the **Ingestion Pipeline**. This process group is responsible for efficiently acquiring raw transaction data from the local filesystem and preparing it for subsequent processing. It consists of three sequentially connected processors:

3.1. ListFile

The ListFile processor continuously monitors the metadata of the designated local source directory (./my_data/input) for any newly arrived CSV files generated by the Python streaming script. It maintains an internal distributed state to track files, ensuring each file is listed exactly once to eliminate redundant tracking. Upon locating a new file, it emits a lightweight FlowFile containing only the object's core attributes and metadata.

3.2. FetchFile

Following the listing phase, the FetchFile processor receives the structural metadata and performs high-performance streaming of the actual physical payload into NiFi's content repository. Upon successful extraction, its completion strategy is set to Delete File from the local source path. This immediate automated cleanup prevents data reprocessing loops and actively preserves server disk space.

3.3. RetryFlowFile

To establish robust error handling and prevent bad data from clogging memory, a specialized RetryFlowFile processor is integrated directly at the output of the FetchFile failure relationships.

- **Operational Logic:** It is explicitly configured to track retry counts via internal flow attributes. It allows a maximum of 3 processing attempts for transient issues.
- **Failure Route:** If a hard error or file corruption persists after the 3rd attempt, the processor routes the failed FlowFile to a dead-letter log relationship, halting further retries. This ensures that a single broken file does not loop infinitely or cause total pipeline starvation.

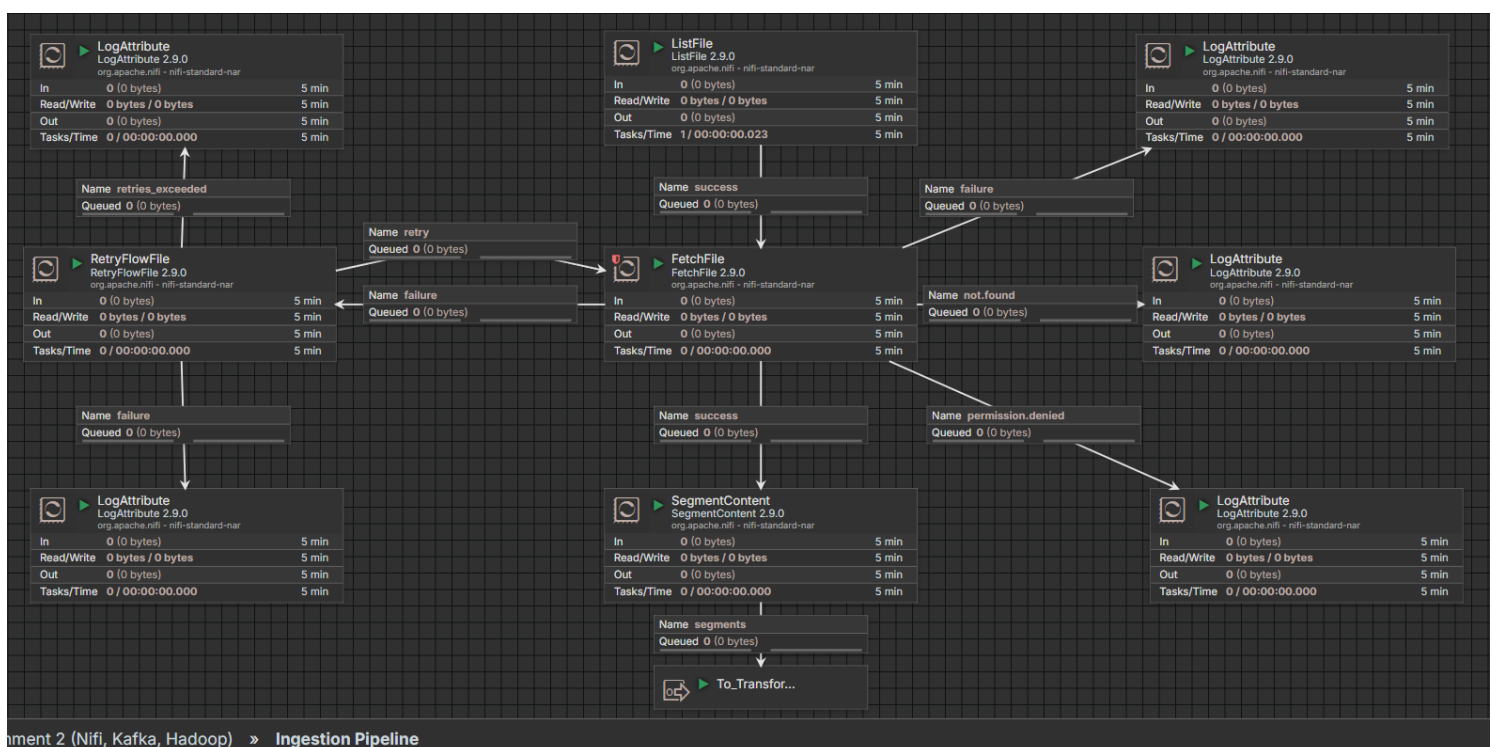
3.4. SegmentContent

To meet strict streaming requirements and prevent memory saturation from oversized payloads, the SegmentContent processor splits large ingested data sheets into uniform micro-chunks. The maximum size for each chunk is restricted to 64 KB. The Keep Line Integrity property is explicitly enforced to guarantee that CSV records are cleanly cut at newline boundaries rather than mid-row.

3.5. Pipeline Optimization and Performance Configurations (Technical Notes)

Rather than acting as separate processing steps, key architectural constraints are configured directly into the processor settings and connection queues to fine-tune performance and stability:

- **Back Pressure Thresholds:** Applied strictly onto all active connection lines. The queue buffers are restricted to a maximum of 10,000 FlowFiles or 1 GB of data capacity. If downstream data transformation delays occur, upstream execution pauses automatically, preventing RAM exhaustion.
- **Prioritization Strategy (FIFO):** All connection queues enforce a strict First In, First Out (FIFO) prioritization scheme. This ensures that banking transactions are processed in the exact order they were discovered, maintaining historical ledger accuracy.
- **Scheduling and Execution Parameters:**
 - **Run Schedule:** Set to 0 sec (Timer-Driven) across the ingestion line to facilitate near-instantaneous, high-frequency execution as soon as threads become available.
 - **Concurrent Tasks:** Configured to 1 to ensure thread safety during sequential file extractions and prevent simultaneous thread collisions on the same block.
 - **Penalty Duration:** Set to 30 sec. If a file hits an I/O exception, it is temporarily penalized, allowing healthy files to bypass it while transient filesystem issues resolve.
 - **Yield Duration:** Configured to 1 sec to pause the processor execution when no new data is present, preventing unnecessary CPU thrashing.



4. Process Group 2: Data Transformation (Parsing & Validation)

The second process group focuses entirely on schema governance, structural normalization, and data quality enforcement. It intercepts the raw 64 KB text chunks emitted by the Ingestion Pipeline, converts them into strict serialized formats, and applies real-time SQL routing logic to isolate injected anomalies before transmission to the message broker.

The pipeline architecture within this process group is designed as follows:

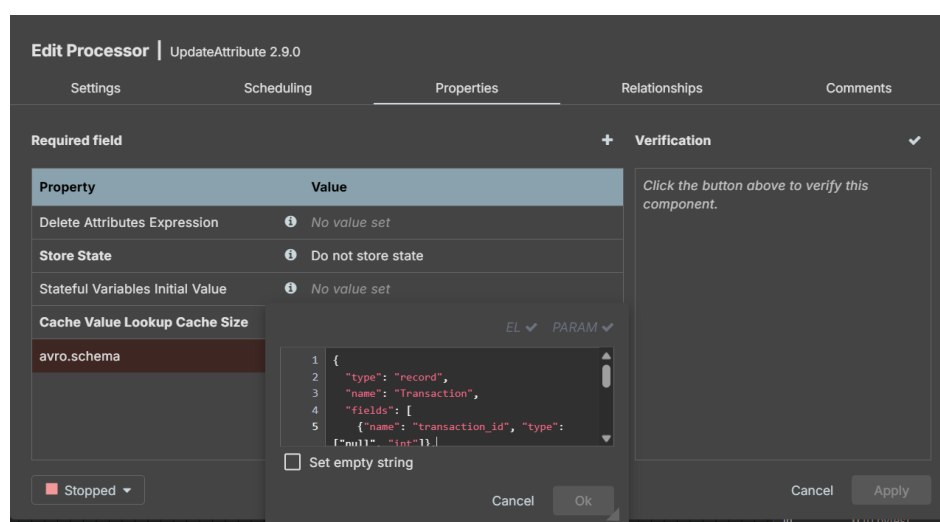
4.1. UpdateAttribute: Avro Schema Injection

The UpdateAttribute processor acts as the entry gateway for data governance. It injects a hardcoded Apache Avro Serialization Schema text directly into each FlowFile's metadata under the attribute key avro.schema. This decoupled architecture allows downstream processors to dynamically adapt to data structures without hardcoding.

To properly accommodate the computational noise and string-based anomalies (such as the word 'invalid') injected by the Python script, the fields are strategically typed as strings or optional types to prevent low-level parsing crashes:

```
{  
  
  "type": "record",  
  "name": "Transaction",  
  "fields": [  
    {"name": "transaction_id", "type": ["null", "int"]},  
    {"name": "customer_id", "type": ["null", "string"]},  
    {"name": "amount", "type": ["null", "string"]},  
    {"name": "timestamp", "type": ["null", "string"]},  
    {"name": "status", "type": ["null", "string"]} ]  
}
```

Note: The amount field is defined as a string union type rather than a native numeric type. This design allows the schema to ingest alphabetical errors safely, deferring numeric type-casting to the validation layer.



4.2. ConvertRecord: CSV to JSON Conversion

The ConvertRecord processor transforms flat-file row chunks into standardized JSON lines. To achieve high-throughput conversion, it orchestrates two foundational Controller Services:

- 1- CSVReader Service: Parses the incoming raw text. It is configured to treat the first line as a header and utilizes the metadata attribute `${avro.schema}` for structural mapping.
- 2- JsonRecordSetWriter Service: Emits the formatted data. Its Output Grouping configuration is set to One Line Per Object, ensuring that records are structured as streamlined JSON arrays optimized for event streaming.

4.3. QueryRecord: Dynamic SQL Routing for Data Quality

The QueryRecord processor executes declarative, memory-efficient SQL routing rules against the live JSON content. It functions as a data firewall, splitting the stream into distinct relationships based on data integrity. It leverages an internal JsonTreeReader to parse the incoming JSON records and a JsonRecordSetWriter to output them.

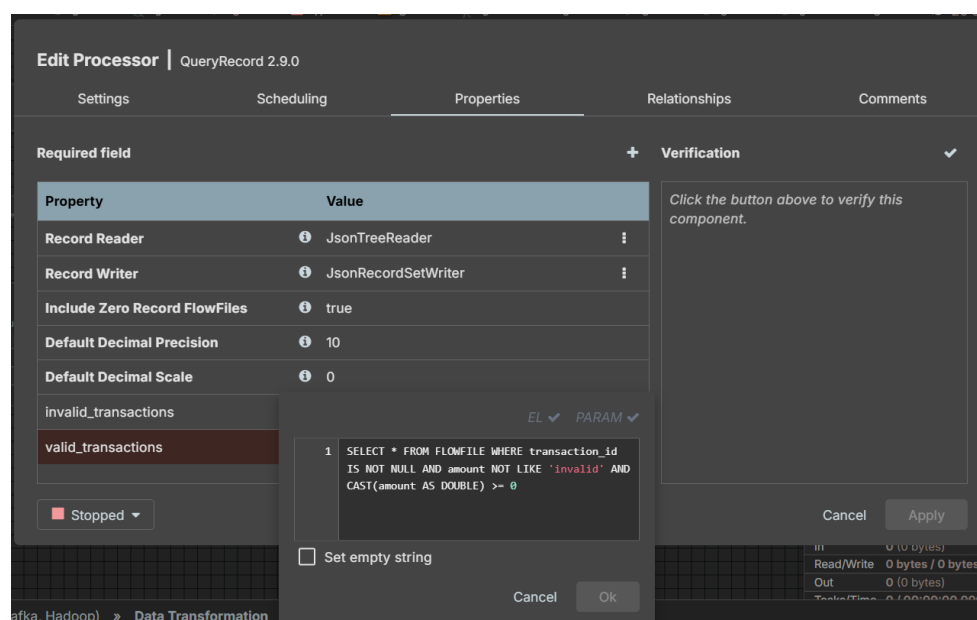
Two explicit, mutually exclusive routing paths are configured to handle validation and anomaly management:

- `valid_transactions` Rule: Routes pristine, actionable financial data downstream.

```
SELECT * FROM FLOWFILE WHERE transaction_id IS NOT NULL AND amount NOT LIKE 'invalid' AND CAST(amount AS DOUBLE) >= 0
```

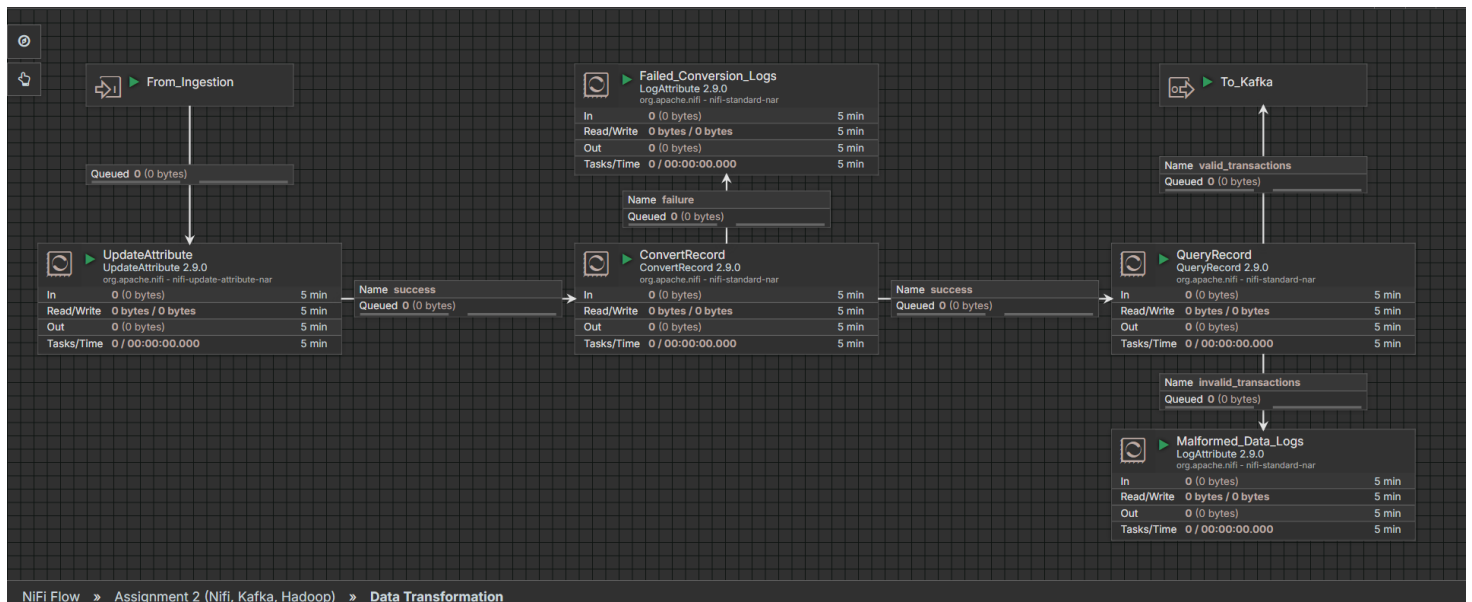
- `invalid_transactions` Rule: Isolates corrupted rows, null identifiers, and mathematical inconsistencies for remediation.

```
SELECT * FROM FLOWFILE WHERE transaction_id IS NULL OR amount = 'invalid' OR CAST(amount AS DOUBLE) < 0
```



4.4. Performance Management & Error Routing (Technical Notes)

- **Stream Consolidation Effect:** Because QueryRecord processes data records line-by-line across multiple chunks, it consolidates individual high-frequency rows. Clean, verified records are merged into unified streams routed to the To_Kafka Output Port, while malformed records are immediately separated.
- **Dead-Letter Queue and Error Logging:** FlowFiles emerging from the invalid_transactions and failure routes are systematically wired to a dedicated LogAttribute processor named Malformed_Data_Logs. This isolates computational noise without blocking the main data channel, ensuring robust pipeline execution.
- **Queue Optimization:** All internal connections enforce First In, First Out (FIFO) prioritization to guarantee chronological sequence integrity, with Back Pressure capped at 10,000 FlowFiles / 1 GB to prevent upstream memory pooling.



5. Process Group 3: Kafka Integration (Distributed Brokering & Queuing)

The third process group is responsible for integrating the validated and transformed data stream with an enterprise Apache Kafka distributed cluster. This stage establishes high-throughput, fault-tolerant message queuing, decoupling the upstream data transformation layer from the downstream storage lake. It enables scalable asynchronous event-driven streaming.

The pipeline infrastructure and command execution within this process group are configured as follows:

5.1. Kafka Topic Creation & Cluster Provisioning

Before any record can be published, a dedicated topic must be provisioned across the distributed cluster infrastructure. The Kafka cluster consists of three active brokers (kafka1:19092, kafka2:19093, and kafka3:19094). To ensure high parallel processing capabilities and absolute data durability, the topic was explicitly created via the secure command-line terminal using the following core infrastructure parameters:

```
kafka-topics --create --topic banking-transactions \  
--bootstrap-server kafka1:19092,kafka2:19093,kafka3:19094 \  
--partitions 3 \  
--replication-factor 2
```

This configuration divides the banking-transactions topic into 3 individual partitions to distribute data traffic evenly and enforce a Replication Factor of 2. This guarantees that every financial transaction record is duplicated across at least two separate brokers, protecting the state against sudden node hardware failures.

```
PS C:\Users\Al-Ariqi Fadi\Documents\Docker\Apache Tools\Kafka> docker compose exec -it kafka1 bash  
[appuser@kafka1 ~]$ kafka-topics --create \  
> --topic banking-transactions \  
> --bootstrap-server kafka1:19092,kafka2:19093,kafka3:19094 \  
> --partitions 3 \  
> --replication-factor 2  
Created topic banking-transactions.  
[appuser@kafka1 ~]$
```

5.2. PublishKafka: Real-Time Event Production

The PublishKafka processor acts as the dedicated producer engine, accepting clean JSON streaming lines from the upstream transformation output. Key configuration properties include:

- **Delivery Guarantee:** Explicitly set to Guarantee Replicated Delivery (equivalent to `acks=all`). This dictates that a transaction record is only marked as successful after being verified and written to all in-sync replicas across the brokers, preventing any data loss.
- **Kafka Key Generation (Dynamic Balancing):** Driven by the evaluation property `${customer_id}`. NiFi dynamically hashes this customer identifier metadata, ensuring that all historical or active banking actions belonging to the exact same client are systematically mapped to the same partition. This preserves absolute sequence ordering for per-customer analytical metrics.
- **Resilience & Retry Handling (The Failure Self-Loop):** To handle transient network timeouts without losing data, the processor's failure relationship is configured with a strict self-loop back onto itself. In the event of a cluster disconnection, the processor queues and retries the data continually. Conversely, the success relationship routes data immediately to a LogAttribute named `Kafka_Publish_Success_Logs` to record successful transmissions.

Edit Processor | PublishKafka 2.9.0

Settings | Scheduling | **Properties** | Relationships

Required field + Verification

Property	Value
Kafka Connection Service	Kafka3ConnectionService
Topic Name	banking-transactions
Failure Strategy	Route to Failure
Delivery Guarantee	Guarantee Replicated Delivery
Compression Type	none
Max Request Size	1 MB
Transactions Enabled	false
Partitioner Class	DefaultPartitioner

Click the button to verify the component.

Stopped

Edit Processor | PublishKafka 2.9.0

Settings | Scheduling | **Properties** | Relationships

Required field + Verification

Property	Value
Partition	No value set
Message Demarcator	No value set
Record Reader	No value set
Record Writer	No value set
FlowFile Attribute Header Pattern	No value set
Kafka Key	\${customer_id}
Kafka Key Attribute Encoding	UTF-8 Encoded
Record Key Writer	No value set

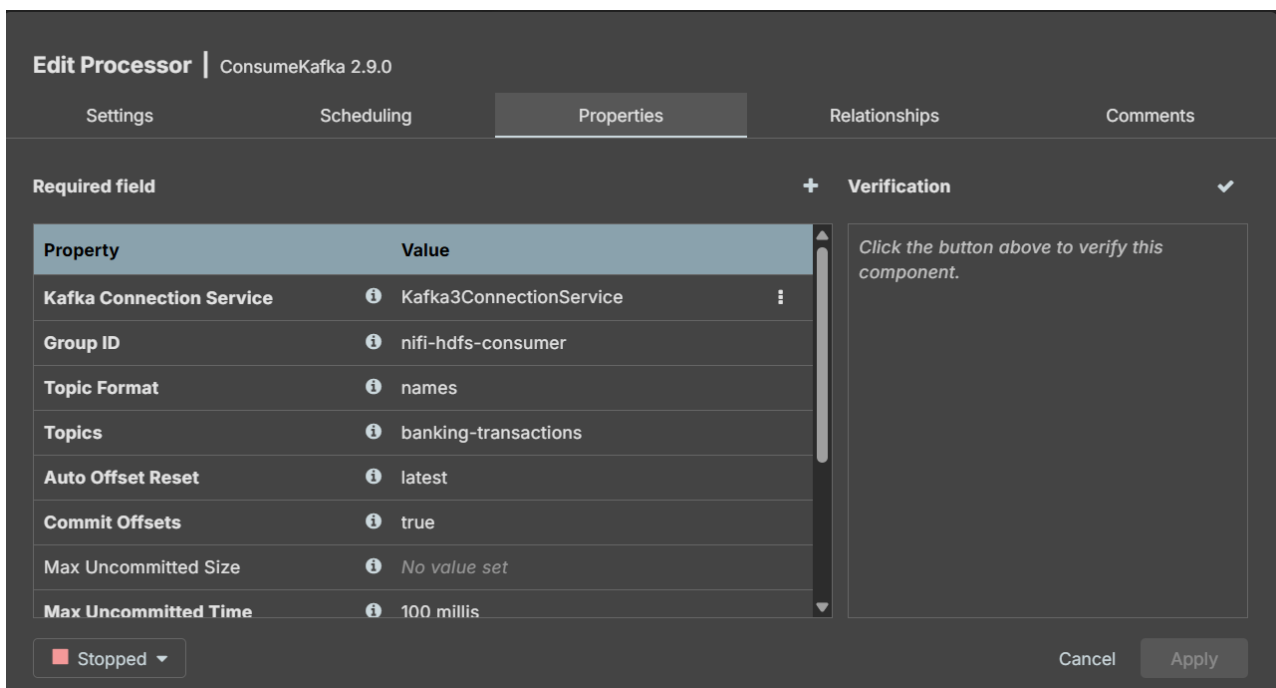
Click the button to verify the component.

Stopped

5.3. ConsumeKafka: Decentralized Event Ingestion

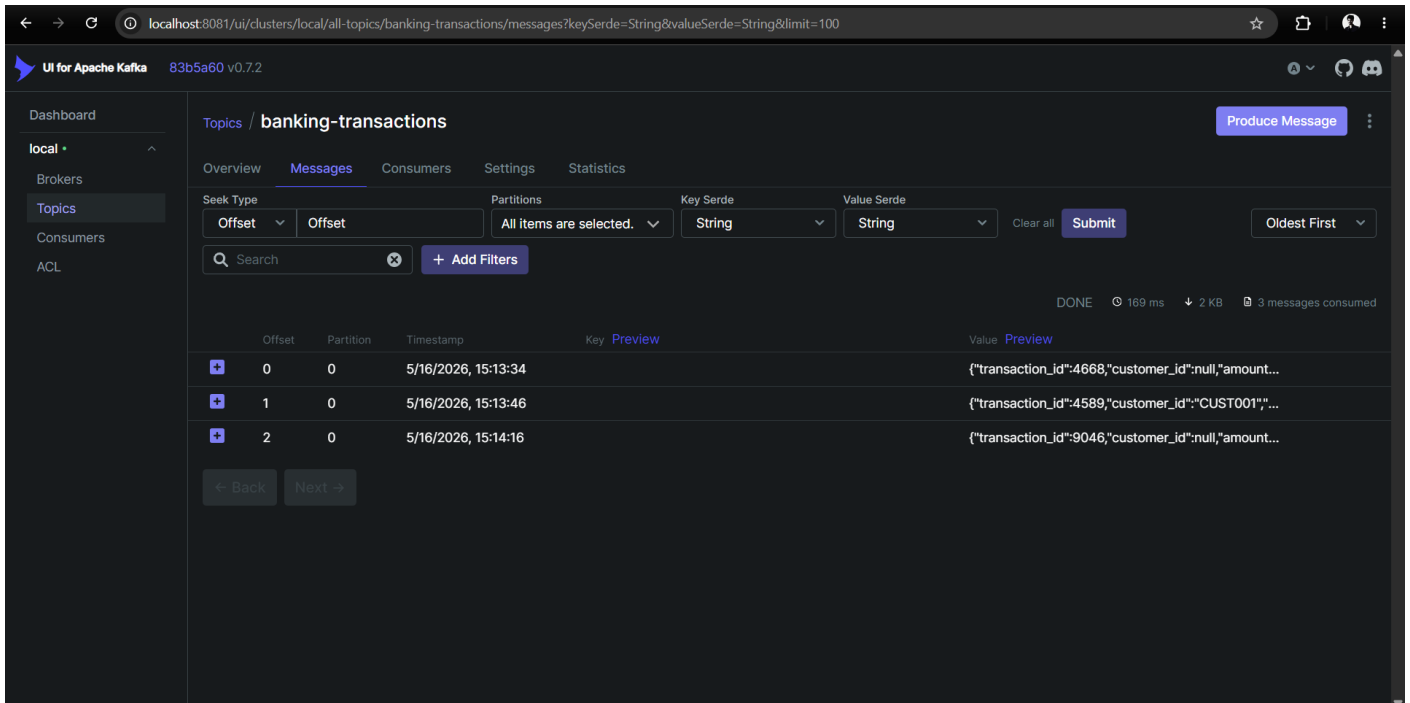
Operating completely independent and decoupled from the PublishKafka processor, the ConsumeKafka processor functions as a background streaming listener. It is bound to only one destination component: an Output Port named `To_HDFS`. This structural separation fulfills true asynchronous architecture. Its primary properties include:

- **Group ID Configuration:** Bound to `nifi-hdfs-consumer`. This registers the processor within Kafka's consumer group protocol, allowing automated offset management and partition rebalancing.
- **Processing Strategy:** Configured to `FLOW_FILE`. This approach optimizes ingestion performance by packing consumed message streams directly into native downstream FlowFiles without introducing structural extraction overhead.



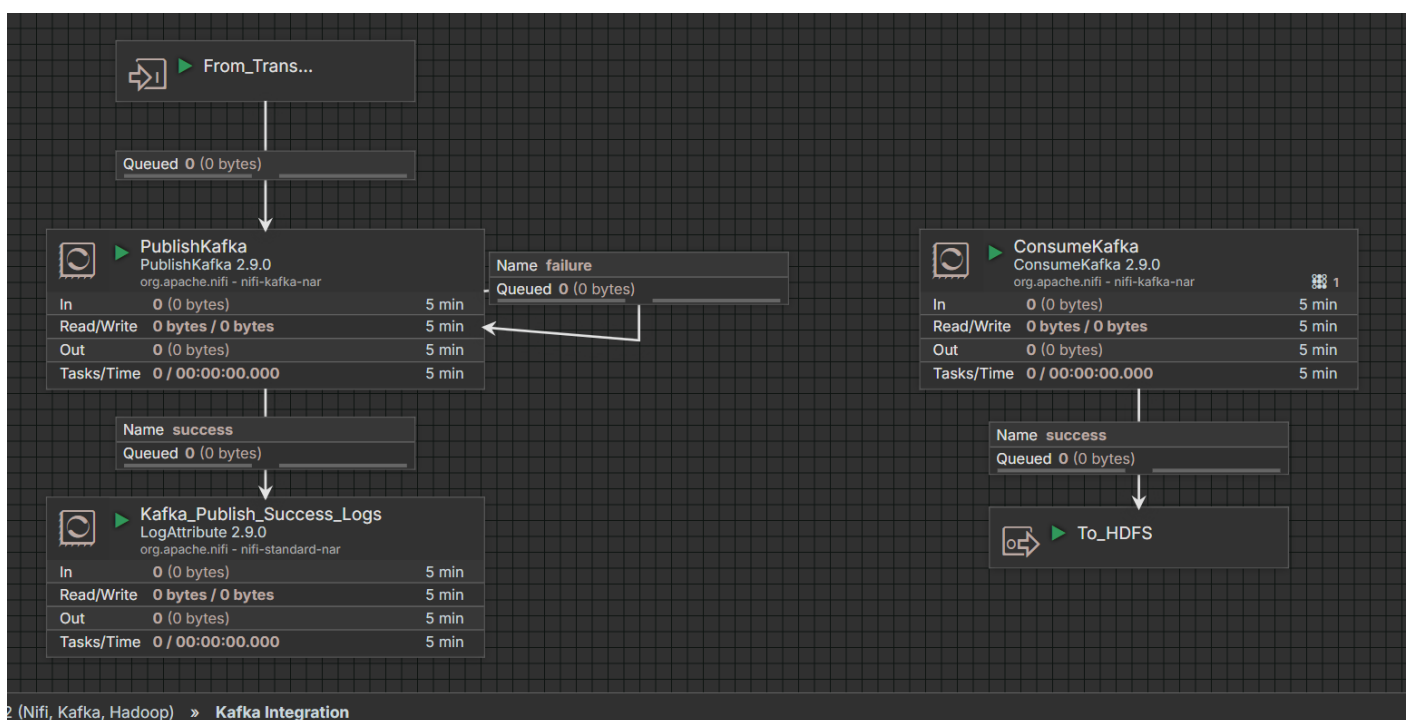
5.4. Verification via Kafka UI Management Dashboard

To provide enterprise observability, data verification is continuously monitored using the interactive Kafka UI Management Dashboard. This administration console provides real-time insights into active partition metrics, consumer group partition lag, broker synchronization parameters, and allows for direct visual payload inspection of the raw JSON objects moving across the cluster.



5.5. Group Performance Optimization & Queue Policies (Technical Notes)

- Back Pressure Tolerances: Internal connection flows between the port and the producer maintain a strict back pressure ceiling of 10,000 FlowFiles / 1 GB. This protects memory stability if a broker partition temporarily blocks threads.
- Queue Prioritization: All queues maintain First In, First Out (FIFO) schema orderings.
- Scheduling Mechanics: The Run Schedule is configured to 0 sec for real-time responsiveness, while the Concurrent Tasks count is set to 1 to preserve chronological state routing into the consumer pipelines.



6. Process Group 4: Hadoop Storage (Analytical Partitioning & Persistence)

The final process group functions as the long-term analytical persistence layer (the Data Lake) of our Big Data pipeline. It is responsible for storing the consumed streaming transaction data into the Hadoop Distributed File System (HDFS). By leveraging HDFS's distributed, fault-tolerant, and horizontally scalable storage capabilities, this stage organizes data to optimize future analytical partitioning and distributed batch querying.

The components and mechanical logic implemented inside this final persistence group are configured as follows:

6.1. UpdateAttribute: Dynamic HDFS Path Generation

An UpdateAttribute processor is positioned at the immediate entrance of this process group to handle metadata synthesis. Rather than writing data into an unstructured folder blob, the pipeline dynamically partitions storage buckets on disk according to the transaction's processing time. This is achieved by utilizing advanced NiFi Expression Language statements to extract exact calendar elements, mapping them into three distinct FlowFile attributes:

- **hdfs.year** attribute statement: `${now():format('yyyy')}`
- **hdfs.month** attribute statement: `${now():format('MM')}`
- **hdfs.day** attribute statement: `${now():format('dd')}`

Constructing metadata attributes dynamically ensures that downstream storage components remain loosely coupled and fully autonomous, reacting instantly to system clock changes.

Edit Processor | UpdateAttribute 2.9.0

Settings

Scheduling

Properties

Relationships

Comments

Required field

+ Verification

✓

Property	Value	
Delete Attributes Expression	No value set	
Store State	Do not store state	
Stateful Variables Initial Value	No value set	
Cache Value Lookup Cache Size	100	
hdfs.day	<code>\${now():format('dd')}</code>	
hdfs.month	<code>\${now():format('MM')}</code>	
hdfs.year	<code>\${now():format('yyyy')}</code>	

Click the button above to verify this component.

Stopped

Cancel

Apply

6.2. PutHDFS: High-Performance Data Lake Persistence

The PutHDFS processor ingests the chronological attributes and streams the JSON transaction content directly into the Hadoop distributed architecture. To achieve seamless network communication and authentication with the remote Hadoop cluster, its properties are strictly configured with the following parameters:

- **Hadoop Configuration Resources:** Explicitly mapped to the cluster's native deployment descriptors: `/opt/nifi/nifi-current/conf/hadoop/core-site.xml` and `/opt/nifi/nifi-current/conf/hadoop/hdfs-site.xml`. This configuration allows the processor to load active NameNode Remote Procedure Call (RPC) addresses and replication rules automatically.
- **Dynamic Target Directory:** The destination path is structured dynamically as follows:

```
/data/year=${hdfs.year}/month=${hdfs.month}/day=${hdfs.day}
```

As time progresses across midnight thresholds, NiFi instantly evaluates the expression language and auto-instantiates new hierarchical storage structures in HDFS without administrative intervention.

- **Conflict Resolution Strategy:** Set explicitly to append. For a continuous, real-time message stream, this strategy ensures that incoming json lines are continuously concatenated into the active file shard. This mitigates the notorious "Small Files Problem" in HDFS, reducing NameNode memory overhead by consolidating streaming data instead of generating thousands of isolated, tiny filesystem blocks.

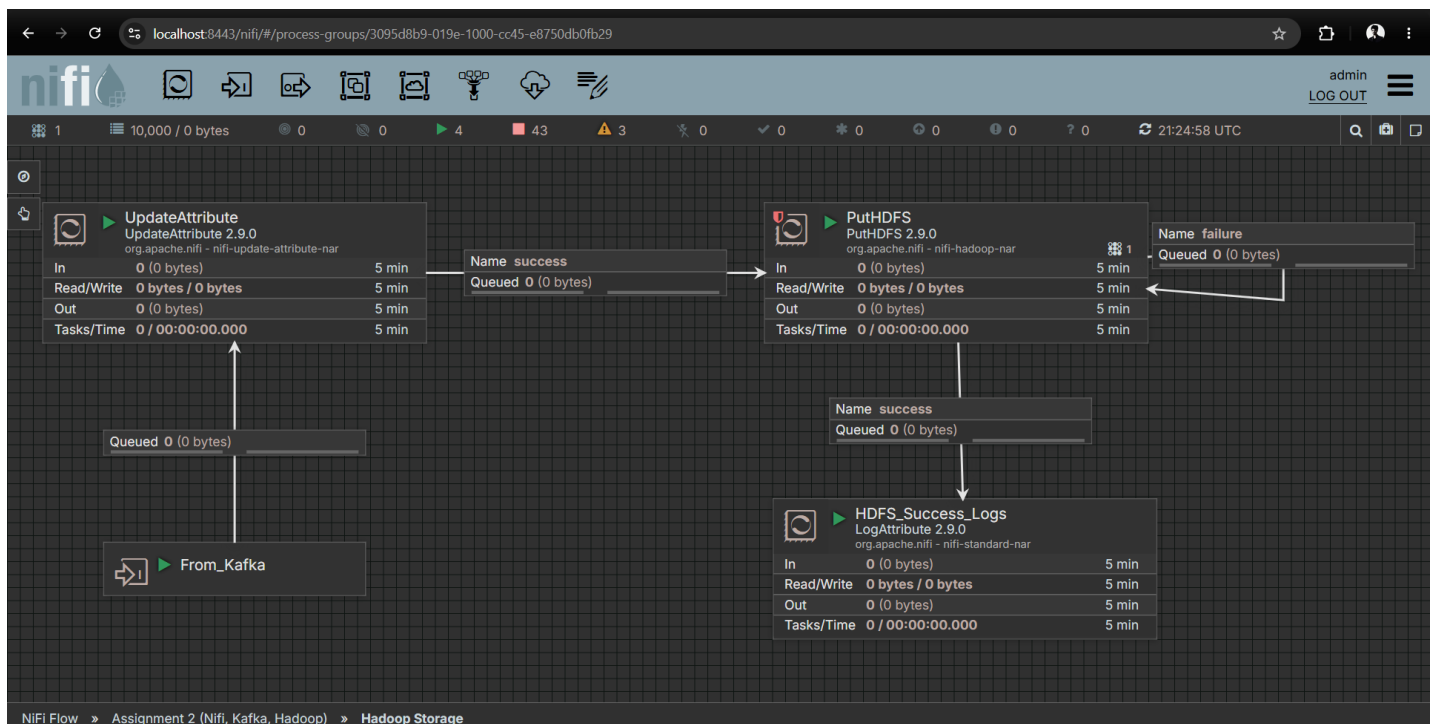
The screenshot shows the 'Edit Processor' window for the 'PutHDFS 2.9.0' processor. The 'Properties' tab is selected, displaying a table of configuration properties. A 'Required field' label is at the top left, and a 'Verification' section with a checkmark icon is at the top right. The properties table includes:

Property	Value
Hadoop Configuration Resources	/opt/nifi/nifi-current/conf/hadoop/core-site.xml, /opt/nifi/nifi-current/conf/hadoop/hdfs-site.xml
Kerberos User Service	
Additional Classpath Resources	No value set
Directory	/data/year=\${hdfs.year}/month=\${hdfs.month}/day=\${hdfs.day}
Conflict Resolution Strategy	append
Append Mode	DEFAULT
Writing Strategy	Write and rename
Block Size	No value set

At the bottom left, there is a status indicator 'Stopped' with a red square icon. At the bottom right, there are 'Cancel' and 'Apply' buttons.

6.3. Pipeline Observability, Validation, and Queue Tuning (Technical Notes)

- **HDFS Generation Verification:** Successful execution is verified by auditing the Hadoop NameNode Web UI filesystem browser (<http://localhost:9870>). The automated structural directory `/data/year=2026/...` is successfully rendered on disk, holding verified clean JSON arrays containing the processed transactions.
- **Fault-Tolerant Error Handling:** The failure and bad records relationships of the PutHDFS processor are safely isolated and wired into a dedicated LogAttribute logging component named `HDFS_Fault_Logs`. This architecture prevents a localized Hadoop I/O timeout or configuration mismatch from causing upstream stalls across the message broker.
- **Queue Mechanics:** Internal connections enforce a First In, First Out (FIFO) schema ordering with Back Pressure safeguards maintained at a 10,000 FlowFiles / 1 GB threshold to protect memory stability during sudden analytical disk write-locks.

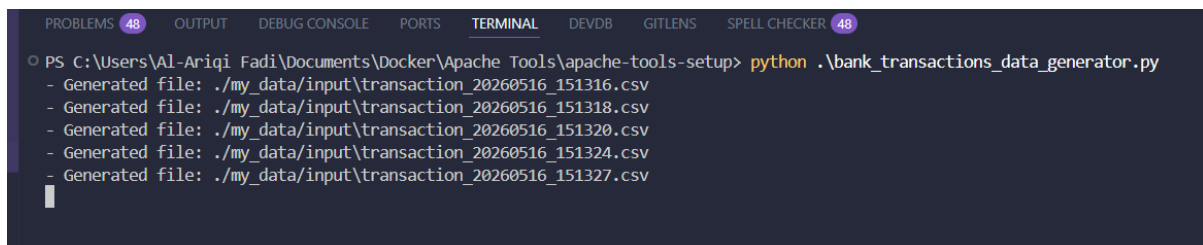


7. End-to-End Pipeline Execution & Deployment Verification

This section provides documented empirical proof of the deployment, orchestration, and continuous live operation of the entire data engineering infrastructure. The following subsections layout the sequential execution verification of each component, mapped directly to their system visual indicators and log states:

7.1. Live Python Data Engine Runtime Execution

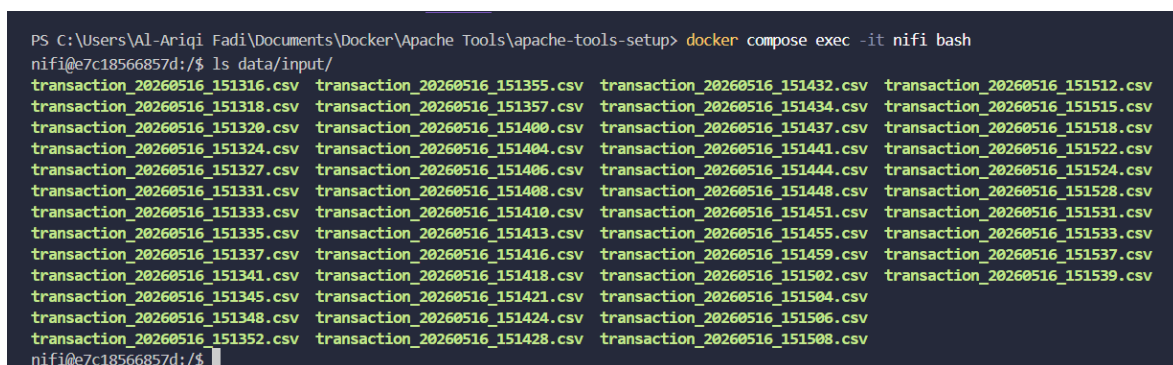
The initialization of the pipeline begins by launching the multi-threaded Python data simulation runtime script within its local isolated storage runtime directory. As shown below, the script successfully instantiates the mock financial events, generating streaming rows and injecting artificial computational noise (duplicates, null values, and custom string errors) while continually outputting new structured flat CSV logs into the local drive paths.



```
PS C:\Users\Al-Ariqi Fadi\Documents\Docker\Apache Tools\apache-tools-setup> python .\bank_transactions_data_generator.py
- Generated file: ./my_data/input/transaction_20260516_151316.csv
- Generated file: ./my_data/input/transaction_20260516_151318.csv
- Generated file: ./my_data/input/transaction_20260516_151320.csv
- Generated file: ./my_data/input/transaction_20260516_151324.csv
- Generated file: ./my_data/input/transaction_20260516_151327.csv
```

7.2. File Container Mapping and NiFi Volume Synchronization

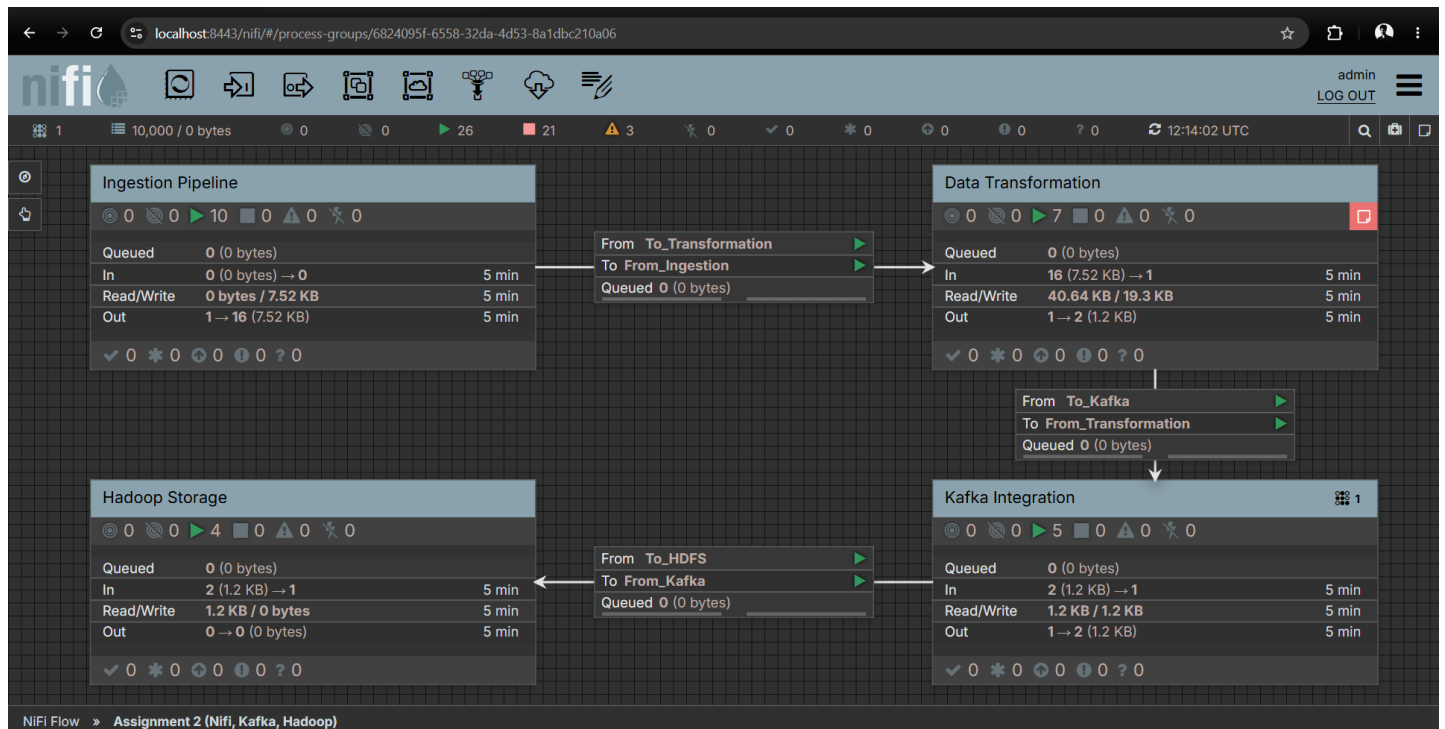
To facilitate direct file system bridging, the deployment architecture utilizes continuous host-to-container volume mapping parameters defined by the storage flag statement: - ./my_data/input:/data/input. The screenshot below verifies that as soon as the Python simulation script flushes a data shard into the host system network folder (./my_data/input), it is instantaneously mirrored, mounted, and made available inside Apache NiFi's isolated internal cluster path (/data/input), allowing the ListFile processor to immediately lock onto the target files.



```
PS C:\Users\Al-Ariqi Fadi\Documents\Docker\Apache Tools\apache-tools-setup> docker compose exec -it nifi bash
nifi@e7c18566857d:/# ls data/input/
transaction_20260516_151316.csv  transaction_20260516_151355.csv  transaction_20260516_151432.csv  transaction_20260516_151512.csv
transaction_20260516_151318.csv  transaction_20260516_151357.csv  transaction_20260516_151434.csv  transaction_20260516_151515.csv
transaction_20260516_151320.csv  transaction_20260516_151400.csv  transaction_20260516_151437.csv  transaction_20260516_151518.csv
transaction_20260516_151324.csv  transaction_20260516_151404.csv  transaction_20260516_151441.csv  transaction_20260516_151522.csv
transaction_20260516_151327.csv  transaction_20260516_151406.csv  transaction_20260516_151444.csv  transaction_20260516_151524.csv
transaction_20260516_151331.csv  transaction_20260516_151408.csv  transaction_20260516_151448.csv  transaction_20260516_151528.csv
transaction_20260516_151333.csv  transaction_20260516_151410.csv  transaction_20260516_151451.csv  transaction_20260516_151531.csv
transaction_20260516_151335.csv  transaction_20260516_151413.csv  transaction_20260516_151455.csv  transaction_20260516_151533.csv
transaction_20260516_151337.csv  transaction_20260516_151416.csv  transaction_20260516_151459.csv  transaction_20260516_151537.csv
transaction_20260516_151341.csv  transaction_20260516_151418.csv  transaction_20260516_151502.csv  transaction_20260516_151539.csv
transaction_20260516_151345.csv  transaction_20260516_151421.csv  transaction_20260516_151504.csv
transaction_20260516_151348.csv  transaction_20260516_151424.csv  transaction_20260516_151506.csv
transaction_20260516_151352.csv  transaction_20260516_151428.csv  transaction_20260516_151508.csv
```


7.3. Centralized Process Group Orchestration (Assignment 2)

The global NiFi architecture is entirely orchestrated through the centralized master canvas group named Assignment 2 (Nifi, Kafka, Hadoop). The overview below captures the global workspace state in an active, running condition. The visual statistics show the continuous, steady influx of incoming data chunks, real-time schema parsing, decoupled messaging broker indexing through the multi-node Kafka cluster topology, and programmatic storage writing loops terminating down inside the storage systems.



7.4. Hadoop HDFS Long-Term Persistence and Audit Verification

Once structural transactions clear the automated message broker consumers, they are dynamically partitioned chronologically and permanently written onto the distributed disk architecture. Successful big data persistence is audited and verified via two distinct administrative control layers:

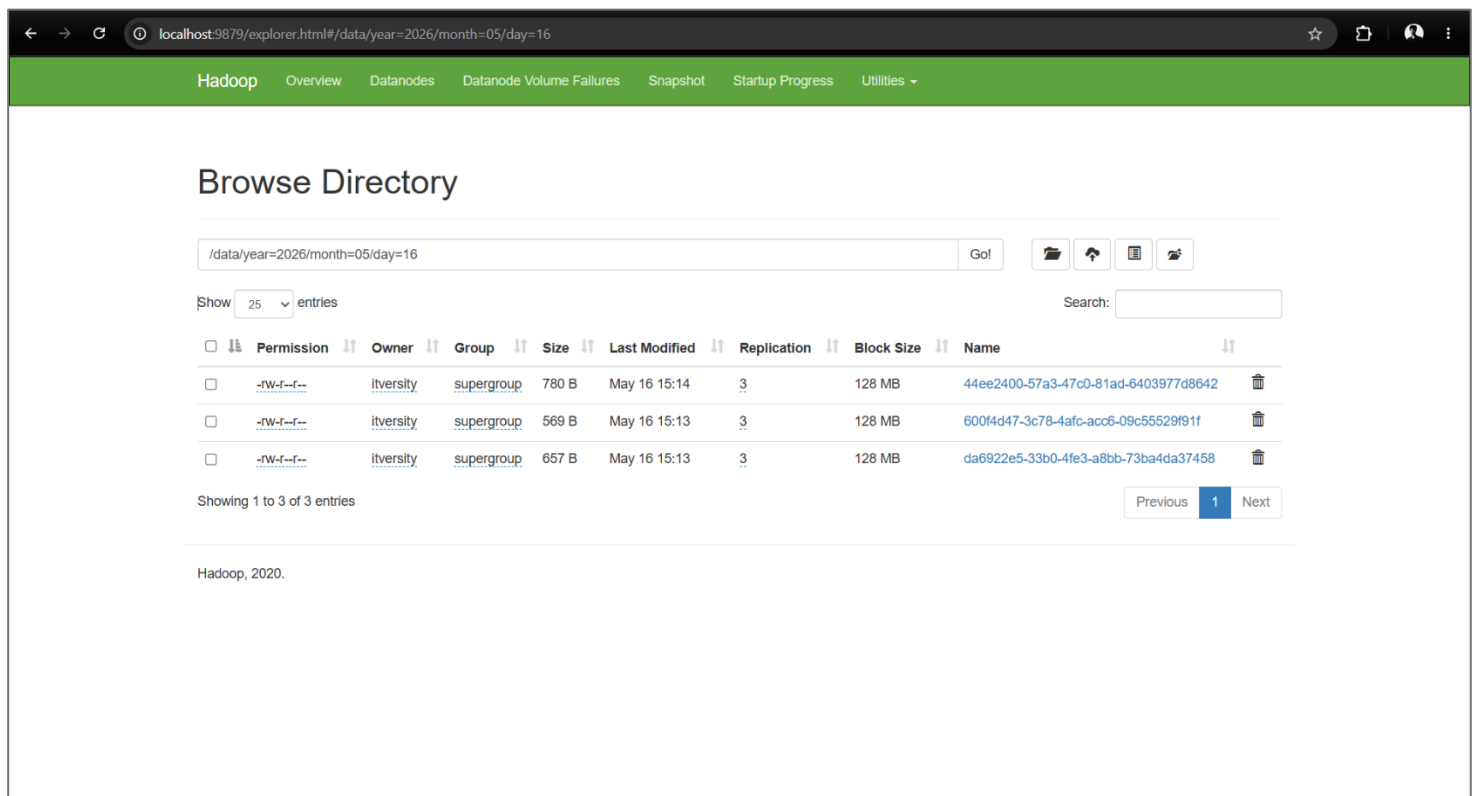
A. Command-Line Interface (HDFS CLI) Storage Audit

The live distributed file system state is queried using terminal orchestration commands (`hdfs dfs -ls /data/...`). The execution returns the verified historical folder tree structures generated dynamically on disk.

```
PROBLEMS 48 OUTPUT DEBUG CONSOLE PORTS TERMINAL DEVDB GITLENS SPELL CHECKER 48
PS C:\Users\Al-Ariqi Fadi\Documents\Docker\Apache Tools\apache-tools-setup> docker compose exec -it itvdelab bash
itversity@itvdelab:~$ hdfs dfs -ls /data
Found 1 items
drwxr-xr-x - itversity supergroup 0 2026-05-16 12:13 /data/year=2026
itversity@itvdelab:~$ hdfs dfs -ls /data/year=2026
Found 1 items
drwxr-xr-x - itversity supergroup 0 2026-05-16 12:13 /data/year=2026/month=05
itversity@itvdelab:~$ hdfs dfs -ls /data/year=2026/month=05
Found 1 items
drwxr-xr-x - itversity supergroup 0 2026-05-16 12:14 /data/year=2026/month=05/day=16
itversity@itvdelab:~$ hdfs dfs -ls /data/year=2026/month=05/day=16
Found 3 items
-rw-r--r-- 3 itversity supergroup 780 2026-05-16 12:14 /data/year=2026/month=05/day=16/44ee2400-57a3-47c0-81ad-6403977d8642
-rw-r--r-- 3 itversity supergroup 569 2026-05-16 12:13 /data/year=2026/month=05/day=16/600f4d47-3c78-4afc-acc6-09c55529f91f
-rw-r--r-- 3 itversity supergroup 657 2026-05-16 12:13 /data/year=2026/month=05/day=16/da6922e5-33b0-4fe3-a8bb-73ba4da37458
itversity@itvdelab:~$
```

B. Web-Based Administration Portal (Hadoop NameNode UI)

To complete enterprise visualization, the screenshot below exhibits the internal directory routing mapping directly inside the Hadoop NameNode Explorer (<http://localhost:9870>). The system displays the fully reconciled storage shards, actively holding the normalized JSON transaction events in their respective time slots.



8. Conclusion & Project Deliverables Manifest

This technical documentation has provided a comprehensive architectural analysis of the design, configuration, and deployment of an end-to-end, high-throughput, real-time data ingestion and processing pipeline. By leveraging the orchestration capabilities of Apache NiFi, the distributed messaging framework of Apache Kafka, and the fault-tolerant cold-storage capacities of Apache Hadoop HDFS, the infrastructure successfully demonstrates modern big data engineering principles. The system capably manages real-time stream synchronization, schema enforcement, programmatic validation, event-driven decoupling, and chronological on-disk partitioning under strict enterprise constraints.

To ensure absolute transparency, verification, and ease of evaluation by the university committee, the final submission package includes all active components of the engineered infrastructure:

- 1- The Python Streaming Engine Source Code: The fully functional background data generation script, containing the transaction algorithms and noise injection logic, explicitly enclosed as an executable script named:
`bank_transactions_data_generator.py`
- 2- The Apache NiFi Global Workspace Blueprint: The complete system configuration and flow architecture of the master canvas "Assignment 2 (Nifi, Kafka, Hadoop)", extracted and exported directly from the environment as a standard JSON configuration descriptor file:
`Assignment_2_(Nifi,_Kafka,_Hadoop).json`

With these components, the entire pipeline can be instantly replicated, verified, and benchmarked across any compliant distributed environment, confirming its architectural robustness, fault-tolerant data routing, and structural resilience against streaming anomalies.